FJSP-DRL 训练框架与预测框架解读

本项目是论文 **"Flexible Job-Shop Scheduling via Graph Neural Network and Deep Reinforcement Learning"**（Song 等）的官方代码：用一个**尺寸无关的异构图神经网络 （HGNN）**作为策略，用 **PPO** 训练，目标是最小化柔性作业车间调度（FJSP）的 makespan （最大完工时间）。

本文梳理训练链路、预测（测试）链路，以及二者共享的模型与环境组件。

目录

- 共享底层组件
- 一、训练框架 train.py
- 二、预测/测试框架 test.py
- 三、模型架构 PPO_model.py + graph/hgnn.py
- 四、每步 env ↔ model 的数据结构
- 整体流程图
- 值得注意的点
- 如何快速掌握整个框架

共享底层组件

配置 config.json（含义见 config_annotation.json）

分四块，训练和测试入口都会加载：

- `env_paras`：作业数 10、机器数 5、`batch_size=20`（并行算例）、工序特征维 6、机器特征维 3、 验证集 `valid_batch_size=100`
- `model_paras`：机器节点 in=3 / out=8，工序节点 in=6 / out=8，`hidden_size_ope=128`，`num_heads=[1,1]`（长度 2 = HGNN 迭代 2 次，每次 1 头），actor/critic 隐层 64
- `train_paras`：`lr=2e-4`、`gamma=1.0`、`K_epochs=3`、`eps_clip=0.2`、 `max_iterations=1000`、`parallel_iter=20`（每 20 轮换一批算例）、`update_timestep=1`、 `save_timestep=10`、`minibatch_size=512`
- `test_paras`：`num_ins=100`、`rules=["DRL"]`、`sample`（决定 DRL-G / DRL-S）、 `num_sample=100`、`num_average=10`、`data_path="1005"`

两个入口在加载配置后都推导出两个关键维度：

```
actor_in_dim  = out_size_ma * 2 + out_size_ope * 2 = 32
critic_in_dim = out_size_ma + out_size_ope         = 16
```

环境注册 env/**init**.py

注册 gym id `fjsp-v0` → `env.fjsp_env:FJSPEnv`。两条链路都用 `gym.make('fjsp-v0', case=..., env_paras=..., data_source=...)`。

数据来源的分叉（关键区别）

- **训练**：`data_source='case'`，由 `CaseGenerator` **实时随机生成**算例
- **验证 / 测试**：`data_source='file'`，从 `.fjs` 文件读入（走 `load_fjs`）

数据加载 env/load_data.py

- `nums_detec(lines)`：解析 FJS 首行得到 `num_jobs`、`num_mas`，并累加每个作业行的首个整数 得到总工序数 `num_opes`
- `load_fjs(lines, num_mas, num_opes)`：把一个 `.fjs` 算例解析成 **8 个张量**（顺序严格对应 `FJSPEnv.__init__` 里的 `tensors[0..7]`）：

位置	变量	形状	含义
0	`matrix_proc_time`	opes×mas	工序在各机器上的加工时间（0 = 不可加工）
1	`matrix_ope_ma_adj`	opes×mas	工序-机器可加工邻接(0/1)
2	`matrix_pre_proc`	opes×opes	直接紧前约束
3	`matrix_pre_proc.t()`	opes×opes	直接紧后约束
4	`opes_appertain`	opes	工序归属哪个作业（末尾补 0 对齐）
5	`num_ope_biases`	jobs	每作业首工序的全局 id（偏移量）
6	`nums_ope`	jobs	每作业的工序数
7	`matrix_cal_cumul`	opes×opes	前驱累积矩阵，第 j 列记录工序 j 沿作业链的全部前驱

`edge_detec()` 是逐作业行的解析器，负责填充上述矩阵。

一、训练框架 train.py

入口 `main()` 流程：

1. 初始化

- 选设备（CUDA / CPU）、加载配置、注入 device
- 构建核心对象：
  - `memories = PPO_model.Memory()` —— rollout 缓冲区
  - `model = PPO_model.PPO(...)` —— 内含 `policy` 与 `policy_old`（都是 `HGNNScheduler`）+ Adam
  - `env_valid = get_validate_env(...)` —— 基于 `./data_dev/{jobs}{mas}/`（如 `./data_dev/1005/`） 全部 100 个文件算例的验证环境
- 输出目录 `./save/train_<时间戳>/`，两个 xlsx 记录验证曲线；最优模型用 `deque(maxlen=1)` 跟踪

2. 主循环 `for i in 1..max_iterations`

1. **每 20 轮换一批算例**：随机采样每作业工序数（范围 `[0.8*num_mas, 1.2*num_mas]`）→ `CaseGenerator` → `gym.make` 新环境（`data_source='case'`，随机合成）
2. **采集 rollout**（`while ~done`）：

    - `actions = model.policy_old.act(state, memories, dones)`（默认 `flag_sample=True, flag_train=True`，即**采样动作并写入 memory**）
    - `state, rewards, dones = env.step(actions)`
    - 把 `rewards`、`dones` 追加进 memory
3. `env.validate_gantt()` 检查甘特图可行性，然后 `env.reset()`
4. **每 1 轮 PPO 更新**：`model.update(memories, ...)` → `memories.clear_memory()`
5. **每 10 轮验证 + 存档**：`validate(...)` 返回平均 makespan，若优于历史最优则删旧存新 `save_best_{jobs}_{mas}_{i}.pt`

3. PPO.update 的内部（PPO_model.py）

- 把 memory 各列堆叠、转置成 `(num_envs, 决策步)` 再 `flatten`，每个决策点变成一个训练样本
- **回报估计不是 GAE**：按环境逆序累加 `discounted_reward = reward + gamma*discounted_reward`（遇终止重置），再按环境做均值/方差归一化。`gamma=1.0`，所以回报 ≈ 原始 reward-to-go， 既当回归目标也当优势基线
- `K_epochs`(3) 内按 minibatch(512) 调 `policy.evaluate(...)` 重算 logprob / state_value / entropy
- `advantages = returns - state_values.detach()`，PPO 裁剪目标：

```
loss = -A_coeff * min(surr1, surr2)
       + vf_coeff * MSE(state_values, returns)
       - entropy_coeff * entropy
```

- K 轮后 `policy_old ← policy`

4. 奖励设计（在 env.step 里）

`reward = 旧makespan - 新makespan`，即每步 makespan 下界估计的下降量。整段 episode 累加后 telescope 成 −最终 makespan，所以**最大化回报 ≈ 最小化 makespan**。

二、预测/测试框架 test.py

入口 `main()` 流程：

1. 加载模型

遍历 `./model/` 下所有 `.pt` 作为待评估"规则"，`torch.load` 后同时载入 `policy` 和 `policy_old`。

2. 逐算例调度

- 读 `./data_test/{data_path}/`（如 `./data_test/1005/`）下 `.fjs` 文件，用 `nums_detec` 按每个算例的实际规模设置 env
- **DRL-S（sample=True）**：`case=[test_file]*num_sample`，一个 env 装 num_sample 份相同算例 并行采样，取 `min` makespan
- **DRL-G（sample=False）**：`case=[test_file]`，贪心，重复 `num_average` 次取均值（结果其实不变）
- `schedule()` 跑完整 episode：`act(..., flag_train=False)` → `env.step`， `flag_train=False` 不写 memory

3. 贪心 vs 采样（在 act 里）

- `flag_sample=True` → `Categorical(action_probs).sample()`（DRL-S）
- `flag_sample=False` → `action_probs.argmax`（DRL-G，确定性）

4. 输出

每算例 makespan + 求解耗时，写到 `./save/test_<时间戳>/makespan_*.xlsx` 和 `time_*.xlsx`， 每个 rule 一列。

validate.py（只被训练用）

- `get_validate_env(env_paras)`：构建基于 `./data_dev/{jobs}{mas}/` 全部算例的文件型环境
- `validate(...)`：本质就是测试循环的复用（固定贪心 `flag_sample=False`、不写 memory `flag_train=False`），返回平均 makespan 和每算例 makespan，用来驱动训练中"最优模型"的选择

三、模型架构 PPO_model.py + graph/hgnn.py

`HGNNScheduler` 是 actor-critic 策略，每次 HGNN 迭代（共 2 次）交替两个阶段：

- **阶段 1 机器嵌入 `GATedge`**：在工序-机器二部图上做带边特征的图注意力。工序 / 机器 / 边 （加工时间）各自线性变换，算注意力系数（`attn_l/attn_r/attn_e`），掩码 softmax（含自环项 `ekk`），sigmoid 输出 `h_mas`，形状 `[B, num_mas, out_size_ma]`
- **阶段 2 工序嵌入 `MLPs`（内含 4 个 `MLPsim`）**：分别对 {机器、紧前工序、紧后工序、自身} 四类邻居做消息传递，拼接后经 `project` MLP → `h_opes`，形状 `[B, max(num_opes), out_size_ope]`。 这里正好用到 `load_fjs` 产出的 `ope_ma_adj / ope_pre_adj / ope_sub_adj`

动作选择 get_action_prob

- 归一化特征（`get_normalized`，验证/DRL-G 时逐算例归一化，因为工序数可能不同）
- HGNN 迭代后池化 `h_mas`（mean）、`h_opes`（mean）
- 每个作业用 `ope_step_batch`（当前工序指针，clamp 到 `end_ope_biases`）gather 出其当前工序嵌入
- 拼接 `[h_job_op, h_machine, h_opes_pooled, h_mas_pooled]` 得 actor 输入 （末维 = `out_size_ma*2 + out_size_ope*2 = 32`），形状 `[B, num_mas, num_jobs, 32]`
- **可行性掩码**：`eligible = job可用 & machine可用 & 该O-M可加工` （由 `mask_job_procing`、`mask_job_finish`、`mask_ma_procing` 组合）， 不可行位置 score 设 `-inf` 再 softmax

act

把展平的动作索引反解回 `(mas, jobs, opes)`：

```
mas  = action_index // num_jobs
jobs = action_index %  num_jobs
opes = ope_step_batch[batch, jobs]
```

返回 `[3, B]` 张量，正好喂给 `env.step`。训练时还会记录 logprob 和 action_index 到 memory。

evaluate（仅 PPO.update 用）

从存储张量重算同样的前向，额外跑 `critic`（`MLPCritic`，输入 `h_pooled`，维度 16）得 state value， 返回 action_logprobs、state_values、dist_entropy。

mlp.py

`MLP / MLPActor / MLPCritic` 是标准多层感知机。Actor/Critic 用 tanh 激活，HGNN 内部的 `MLPsim / project` 用 ELU。

四、每步 env ↔ model 的数据结构

ENV → MODEL：`EnvState`（fjsp_env.py 的 @dataclass）

- **静态图**：`ope_ma_adj_batch [B,O,M]`、`ope_pre_adj_batch [B,O,O]`、`ope_sub_adj_batch [B,O,O]`、 `opes_appertain_batch`、`end_ope_biases_batch`、`nums_opes_batch`
- **动态特征**：`feat_opes_batch [B,6,O]`（状态 / 邻接机器数 / 加工时间 / 未排工序数 / 完工时间 / 开始时间）、`feat_mas_batch [B,3,M]`（邻接工序数 / 可用时间 / 利用率）、 `proc_times_batch [B,O,M]`
- **动态掩码/指针**：`mask_job_procing_batch`、`mask_job_finish_batch`、`mask_ma_procing_batch`、`ope_step_batch`（当前工序指针）、`batch_idxes`、`time_batch`

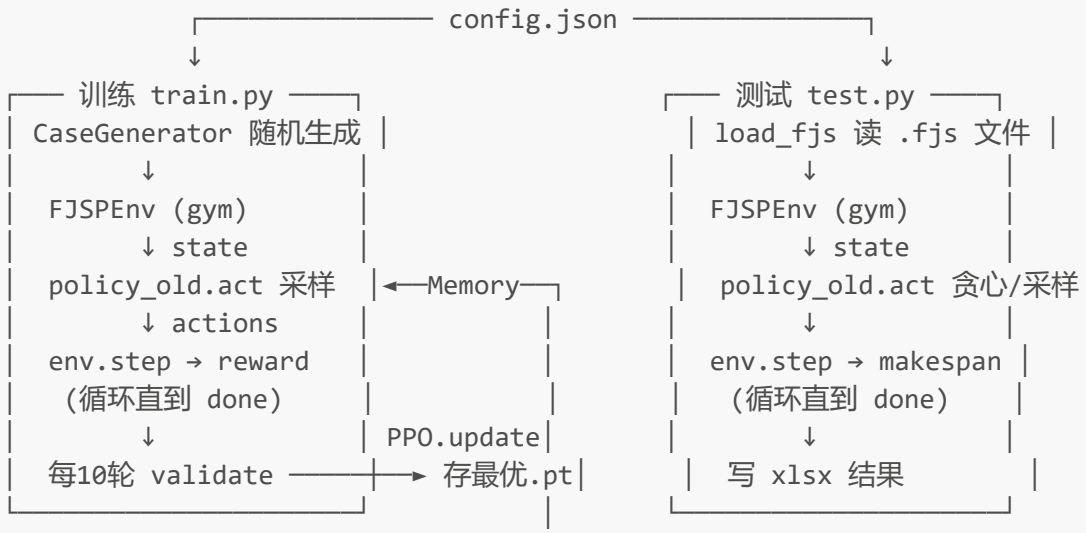
MODEL → ENV

`actions [3, B] = (opes, mas, jobs)`，由 `FJSPEnv.step()` 消费。

env.step() 返回 (state, reward, done)

落实 O-M 分配、裁剪未用弧、用 `cal_cumul_adj_batch`（`torch.bmm`）重算工序起止时间、更新机器 可用时间 / 利用率、更新掩码和指针、算 reward，再通过 `if_no_eligible() / next_time()` 推进仿真 时间直到有可行 O-M 对或算例完成。batch 是半同步的：`batch_idxes` 随算例完成而收缩 （`mask_finish` 对比 `N` 与 `nums_opes`）。

整体流程图



```
                └──── 每1轮 PPO 更新策略 ────┘                ↑
                                                        加载 ./model/*.pt
```

值得注意的点

- **gamma=1.0 + 回报归一化**：是"价值基线式优势"，不是 GAE；makespan 递减奖励会 telescope 成 −makespan
- **逐算例归一化/池化**只在 `flag_sample=False and flag_train=False`（验证 / DRL-G）时启用， 因为文件 batch 里各算例工序数可能不同；训练和 DRL-S 假设工序数一致
- `test.py` 用 `mod_files[i_rules]` 索引 checkpoint，依赖 rules 顺序和 `./model/` 文件顺序对齐， 单个 `.pt` 没问题，多个会脆弱
- 工序全程用**全局 id**（靠 `num_ope_biases` 偏移），动作反解和 `ope_step` 指针都依赖这一点
- `load_fjs` 的 `line is "\n"` 用的是身份比较而非值比较，仅靠 CPython 字符串驻留碰巧能工作， 建议改成 `line == "\n"`

如何快速掌握整个框架

核心原则：先跑通一个最小闭环，再逐层拆。 不要一上来通读 `PPO_model.py` 的每一行 （那是最难的部分，缺上下文会很痛苦）。应从**数据和环境**这端往上爬，因为它们具体、可观察。

下面按"投入产出比"排序给出一条学习路径。

第 1 步：盯住"一步"而不是"全局"（约半天）

整个框架的心脏就一句话循环，训练和测试都是它：

```
actions = model.policy_old.act(state, memories, dones)   # 模型看状态 → 出动作
state, rewards, dones = env.step(actions)                # 环境执行 → 新状态+奖励
```

先彻底搞懂**一次 step 里发生了什么**，比读 1000 行代码都管用。做法：

- 用 `C:\Python\python310` 作为解释器，在 `train.py` 的 `while ~done` 循环里打断点
- 单步观察三样东西的形状和值：`state`（`EnvState` 的字段）、`actions`（`[3,B]`）、`rewards`
- 重点确认：动作索引如何反解成 `(机器，作业，工序)`（`PPO_model.act` 里的取模/整除三行）

做完这一步，就从"看代码"变成"看数据流动"，框架立刻具象化。

第 2 步：按数据维度画一张"张量流转图"（约半天）

RL + GNN 代码最难的不是逻辑，而是**一堆形状相近的张量分不清谁是谁**。边断点边核对关键张量：

张量	形状	一句话
`feat_opes_batch`	`[B, 6, O]`	工序特征
`feat_mas_batch`	`[B, 3, M]`	机器特征

张量	形状	一句话
`ope_ma_adj_batch`	`[B, O, M]`	工序-机器图
`h_opes` / `h_mas`	`[B, O, 8]` / `[B, M, 8]`	HGNN 嵌出
`action_probs`	`[B, M*J]`	每个 O-M 对的概率
`actions`	`[3, B]`	选中的 (opes, mas, jobs)

本文档上面已给出这些，但**自己对着断点验证一遍**记忆才牢。

第 3 步：用"改一个变量看结果"验证理解（约 1 天）

掌握框架最快的方式是**破坏它然后观察**。几个高性价比的小实验：

- 把 `act` 里的 `argmax` 换成随机动作 → 看 makespan 是否明显变差（验证"策略确实在学"）
- 把 `config.json` 的 `num_jobs/num_mas` 改成 `6/3` 跑几轮 → 验证"尺寸无关"是真的
- 在 `env.step` 里 print 出 `reward_batch` → 看它是否每步 ≤ 0、累加 ≈ −makespan（验证奖励设计）

第 4 步：最后才啃 HGNN 和 PPO.update（约 1-2 天）

这两块理论最密，放最后，因为前三步已给了足够上下文：

- **HGNN**（`graph/hgnn.py`）：只需理解"机器嵌入 ← 它能连的工序；工序嵌入 ← 它的前驱/后继/机器"。`GATedge` 的注意力细节可先跳过
- **PPO.update**：抓住三件事——回报怎么算（逆序累加 reward-to-go）、`advantage = returns − value`、clip 目标。不要纠结每个 reshape

学习顺序一览

```
数据（load_fjs / .fjs）
   ↓  具体、可打印
环境（FJSPEnv.step 的一次转移）
   ↓  一句话循环：act → step
动作反解（act 里的 index → opes/mas/jobs）
   ↓  张量形状对齐
模型前向（HGNN 两阶段嵌入 → actor 打分）
   ↓  理论最密，放最后
训练更新（PPO.update 回报/优势/裁剪）
```

> 建议：先做一个"最小运行脚本"（剥离 visdom / xlsx / 多环境等干扰，只保留 生成 1 个算例 → 跑一次完整调度 → 打印每步 state/action/reward），边跑边断点， 半天即可摸清主干。